

JAVA PROGRAMMING

Chapter 7: Loops in Java

By Akanshu Jamwal

Table of Contents

1. What Are Loops?
2. Why Do We Need Loops?
3. for Loop - Detailed Guide
4. while Loop - Detailed Guide
5. do-while Loop - Detailed Guide
6. while vs do-while Comparison
7. Nested Loops
8. break Keyword in Loops
9. continue Keyword in Loops
10. for vs while vs do-while
11. Infinite Loops
12. Common Mistakes & Prevention

1. What Are Loops?

A loop is a control structure that **repeats a block of code** as long as a condition is satisfied.

Java provides 3 main loops:

- for loop
- while loop
- do-while loop

2. Why Do We Need Loops?

Loops help in:

- Reducing code repetition
- Writing shorter code
- Solving pattern problems

3. for Loop - Detailed Guide

The for loop is used when you **know in advance** how many times the loop should run.

Syntax:

```
for (initialization; condition; update) {  
    // code  
}
```

Example: Print 1 to 5

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}  
// Output: 1 2 3 4 5
```

4. while Loop - Detailed Guide

Used when **number of iterations is not fixed**. It keeps running while condition is true.

Syntax:

```
while (condition) {  
    // code  
}
```

Example:

```
int count = 1;  
while (count <= 5) {  
    System.out.println(count);  
    count++;  
}
```

5. do-while Loop - Detailed Guide

Executes body at least once, even if condition is false initially.

Syntax:

```
do {  
    // code  
} while (condition);
```

Note: Semicolon after while(condition) is important!

6. while vs do-while Comparison

Aspect	while	do-while
Check condition	Before execution	After execution
Minimum runs	0 times	1 time

7. Nested Loops

A loop inside another loop. Used for patterns, matrices, and grids.

Example: 3x3 Pattern

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 3; col++) {
        System.out.print("* ");
    }
    System.out.println();
}
// Output:
// * * *
// * * *
// * * *
```

8. break Keyword in Loops

The break keyword **immediately stops the loop**.

```
for (int i = 1; i <= 6; i++) {
    if (i == 4) {
        break; // Loop stops
    }
    System.out.println(i);
}
// Output: 1 2 3
```

9. continue Keyword in Loops

The continue keyword **skips current iteration** and moves to next.

```
for (int day = 1; day <= 5; day++) {
    if (day == 3) {
        continue; // Skip 3
    }
    System.out.println(day);
}
// Output: 1 2 4 5
```

10. for vs while vs do-while

Loop	Best For
for	Fixed repetitions, arrays
while	Condition-based repetitions
do-while	Must run at least once

11. Infinite Loops

If condition **never becomes false**, loop runs forever.

```
int num = 1;
while (num <= 5) {
    System.out.println(num);
    // num++ is missing - INFINITE!
}
```

12. Common Mistakes & Prevention

■ Mistake 1: Forgetting update statement

- Missing i++ causes infinite loop
- Wrong condition (i >= 5 when i = 1)
- Extra semicolon after loop statement
- Confusing while and do-while behavior
- Same variable name in nested loops

■ Key Takeaway:

Loops + Conditionals = Foundation of Programming

Loop mastery bridges basic syntax to problem solving. Master this deeply. Don't rush!